



Heaps e Heapsort



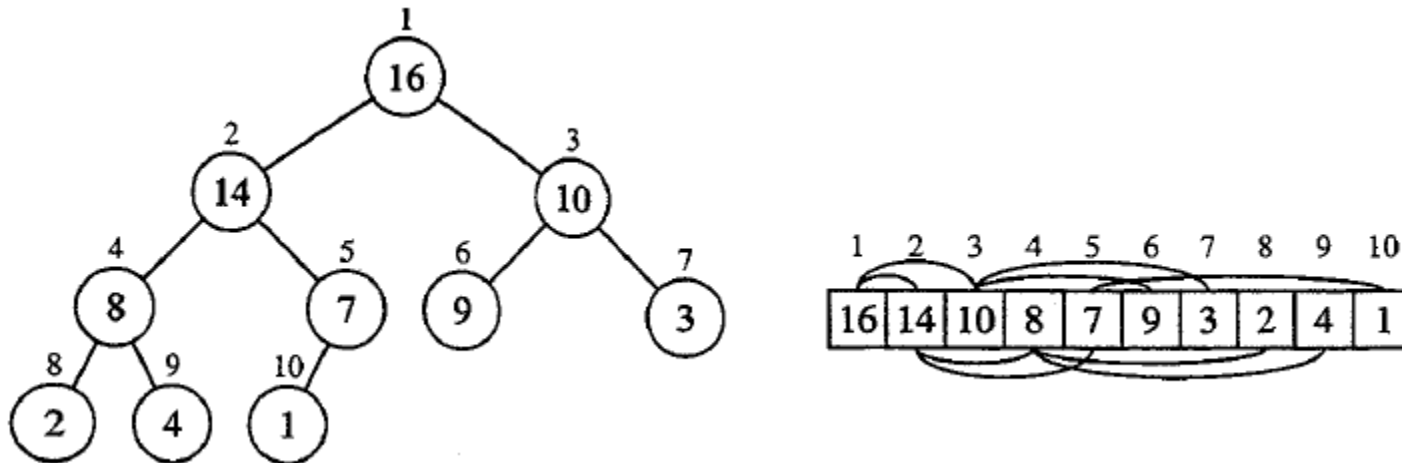
Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Heaps
 - ▣ Definição
 - ▣ Construção
- Heapsort

Heaps

- Um arranjo (*array*) de elementos que pode ser visto como uma árvore binária praticamente completa
- ▣ Último nível deve estar preenchido da esquerda para a direita até certo ponto



Array representando os elementos de um heap. Figura retirada de Cormen [1]

Heaps

- O arranjo que representa um heap possui dois atributos:
 - ▣ Comprimento: número de posições no array
 - ▣ Tamanho_do_heap: número de elementos do heap armazenado no array
 - ▣ Ou seja, $\text{tamanho do heap} \leq \text{comprimento}$
 - ▣ Elementos do array de índice maior que o tamanho do heap fazem parte do arranjo, mas não do heap

Heaps

```
typedef struct heap {
    int* vetor;
    int tamanho_heap;
    int comprimento;
}Heap;

Heap* cria(int comprimento)
{
    Heap* heap = (Heap*)malloc(sizeof(Heap));
    heap->comprimento = comprimento;
    heap->vetor = (int*)malloc(comprimento * sizeof(int));
    heap->tamanho_heap = 0;
    if (!heap->vetor)
    {
        printf("estouro de memoria na criacao do heap");
        return NULL;
    }
    return heap;
}
```

Heaps

- Raiz da árvore

 - ▣ Encontrado na primeira posição do array (*array*[0])

- Acessando os filhos e o pai do i — *ésimo* nó:

```
int esquerda(int indPai)
{
    return 2 * indPai;
}
```

```
int direita(int indPai)
{
    return 2 * indPai + 1;
}
```

```
int pai(int indPai)
{
    return indPai/2;
}
```

Heaps

□ Dois tipos de heaps

▣ Heap máximo

- Valor de um nó não pode ser maior que o valor do seu pai
- Maior elemento de um heap máximo está localizado na raiz
 - Propriedade válida para todas as subárvores de uma árvore

▣ Heap mínimo

- O oposto
- Valor de um nó não pode ser menor que o do seu pai
- Menor elemento localizado na raiz

Heaps

□ MAX-HEAPIFY

- Mantem a propriedade da heap
- Entradas: *array* representando a heap e um índice i do array representando um nó da heap
- Procedimento supõe que filhos à direita e à esquerda de i são heaps máximos
- Caso o elemento em i viole a propriedade da heap (caso seja menor que um dos seus filhos)
 - Elemento irá “descer” na árvore, sendo posicionado no local correto
 - Subárvore no índice i se torna um heap máximo

Heaps

□ MAX-HEAPIFY

- Compara o valor de um nó com o dos seus filhos
- Caso apenas um dos filhos possua valor maior ele troca de posição com o pai
- Caso os dois sejam maiores, o maior deles troca de posição com o pai
 - A troca pode fazer com que o filho viole a propriedade da heap
 - Então `max_heapify` é recursivamente chamada para a posição para onde o nó avaliado se moveu (antigo filho de maior valor)
- Caso nenhum filho seja maior, nenhuma troca ocorre e o procedimento se encerra

Heaps

- Acessando uma posição no array

```
int at(Heap* heap, int indice)
{
    return heap->vetor[indice - 1];
}
```

Heaps

```
void max_heapify(Heap* heap, int indPai)
{
    int indEsq = esquerda(indPai);
    int indDir = direita(indPai);
    int maior = indPai;
    if (indEsq <= heap->tamanho_heap && at(heap, indEsq) > at(heap, indPai))
    {
        maior = indEsq;
    }
    if (indDir <= heap->tamanho_heap && at(heap, indDir) > at(heap, maior))
    {
        maior = indDir;
    }
    if (maior == indPai)
        return;

    int aux = at(heap, maior);
    heap->vetor[maior-1] = at(heap, indPai);
    heap->vetor[indPai-1] = aux;
    max_heapify(heap, maior);
}
```

Testando a função max_heapify

□ Funções de inserção e impressão

```
void insere(Heap* heap, int valor)
{
    if (heap->tamanho_heap >= heap->comprimento )
    {
        return;
    }

    heap->vetor[heap->tamanho_heap++] = valor;
}

void imprime(Heap* heap)
{
    printf("\n");
    for (int i = 0; i < heap->tamanho_heap; i++)
        printf("%d ", heap->vetor[i]);
}
```

Testando a função max_heapify

```
int main()
{
    Heap* heap = cria(10);

    insere(heap, 16);
    insere(heap, 4); insere(heap, 10);
    insere(heap, 14); insere(heap, 7); insere(heap, 9); insere(heap, 3);
    insere(heap, 2); insere(heap, 8); insere(heap, 1);

    heap->tamanho_heap = 10;
    imprime(heap); //16 4 10 14 7 9 3 2 8 1
    max_heapify(heap, 2);
    imprime(heap); //16 14 10 8 7 9 3 2 4 1
    getchar();
    return 0;
}
```

Análise max_heapify

- Tempo para corrigir a posição de um determinado nó, trocando-o ou não com um dos seus filhos
 - ▣ $\theta(1)$
- Tempo para aplicar recursivamente o max_heapify em uma das subárvores
 - ▣ Altura da árvore
 - ▣ $O(\log(n))$

Heap

□ Construção da Heap

- Usa-se a função `max_heapify` de baixo pra cima

- A partir do elemento de posição $\frac{\text{tamanho_heap}}{2}$ até a raíz

- Todos os elementos maiores que $\frac{\text{tamanho_heap}}{2}$ são folhas

- E, portanto, não violam o critério da heap

Heap

□ Construção da Heap

```
void build_max_heap(heap* heap)
{
    int posInicial = heap->tamanho_heap / 2;
    for (int i = posInicial; i >= 1; i--)
    {
        max_heapify(heap, i);
    }
}
```

Heap

□ Testando construção da Heap

```
int main()
{
    Heap* heap = cria(10);

    insere(heap, 1);
    insere(heap, 2); insere(heap, 3);
    insere(heap, 4); insere(heap, 5); insere(heap, 6); insere(heap, 7);
    insere(heap, 8); insere(heap, 9); insere(heap, 10);

    heap->tamanho_heap = 10;
    imprime(heap); //1 2 3 4 5 6 7 8 9 10
    build_max_heap(heap);
    imprime(heap); //10 9 7 8 5 6 3 1 4 2
    getchar();
    return 0;
}
```

Análise construção da heap

- Limite superior

- Max_heapify: $O(\log(n))$

- Para um heap com n nós, temos o limite superior de $O(n \log(n))$

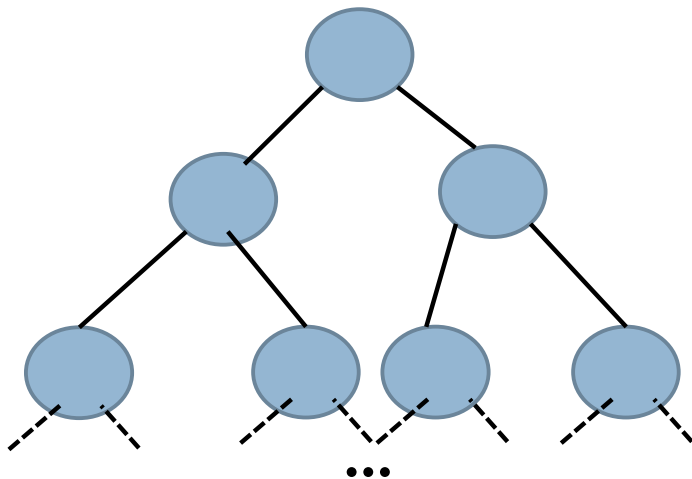
- No entanto, esse limite superior não é assintoticamente restrito

- Observação: tempo de max_heapify varia com a altura de cada nó e a altura da maioria dos nós é pequena

Análise construção da heap

- Limite assintoticamente restrito
 - ▣ Custo do build_heap por nível:

numero de nós × altura



$$1 \times h$$

$$2 \times (h - 1)$$

$$4 \times (h - 2)$$

Generalizando, para uma árvore de altura h , subindo j níveis da base para o topo, temos o custo em cada nível como: $2^{h-j} \times j$, $j = 0 \dots h$



$$n \times 0$$

Análise construção da heap

□ Limite assintoticamente restrito

- Custo do build_heap somando todos os níveis, somando da base para o topo:

- $$T(n) = \sum_{j=0}^h j 2^{h-j} = \sum_{j=0}^h j \frac{2^h}{2^j} = 2^h \sum_{j=0}^h \frac{j}{2^j} \quad (1)$$

- Sabendo que

- $$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x} \quad (2)$$

- Derivando (2) dos dois lados temos

- $$\sum_{j=0}^{\infty} j x^{j-1} = \frac{1}{(1-x)^2}$$
- $$\sum_{j=0}^{\infty} j x^j = \frac{x}{(1-x)^2} \quad (3)$$

Análise construção da heap

□ Limite assintoticamente restrito

$$□ \sum_{j=0}^{\infty} jx^j = \frac{x}{(1-x)^2} \quad (3)$$

□ Substituindo $x = \frac{1}{2}$ em (3), temos

$$□ \sum_{j=0}^{\infty} \frac{j}{2^j} = \frac{1/2}{(1-1/2)^2} = \frac{1/2}{1/4} = 2$$

□ Assim, a partir de (1), temos:

$$□ T(n) = 2^h \sum_{j=0}^h \frac{j}{2^j} \leq 2^h \sum_{j=0}^{\infty} \frac{j}{2^j} \leq 2^h \cdot 2$$

$$□ T(n) \leq 2^{h+1}$$

Análise construção da heap

- Limite assintoticamente restrito
 - ▣ $T(n) \leq 2^{h+1}$
 - ▣ Como o número máximo de nós em uma árvore binária de altura h é $n = 2^{h+1} - 1$, temos que
 - ▣ $T(n) \leq n + 1 \in O(n)$
 - ▣ Além disso, o algoritmo também possui complexidade $\Omega(n)$, já que todos os nós da árvore precisam ser acessados
 - Assim podemos afirmar que a construção da heap possui complexidade de tempo $\Theta(n)$

Heapsort

- Constrói um heap a partir de um array a ser ordenado
- Elemento máximo encontra-se na primeira posição do heap ($array[1]$)
 - ▣ Este pode ser colocado em sua posição correta
 - Trocando o primeiro pelo último elemento $array[1] \leftrightarrow array[n]$
 - Descartando o último nó (antiga raiz), podemos facilmente transformar o novo arranjo $array[0 \dots n - 1]$ novamente em um heap
 - $max_heapify(heap, 1)$
 - $array[0 \dots n - 1]$ torna-se um heap máximo!
 - Processo é então repetido para $array[0 \dots n - 1]$

Heapsort

```
void heapsort(Heap* heap)
{
    build_max_heap(heap);

    for (int i = heap->tamanho_heap; i >= 2; i--)
    {
        troca(heap, 1, i);
        heap->tamanho_heap--;
        max_heapify(heap, 1);
    }
}

void troca(Heap* heap, int indice1, int indice2)
{
    if (indice1 > heap->tamanho_heap || indice2 > heap->tamanho_heap)
    {
        return;
    }
    int aux = heap->vetor[indice1 - 1];
    heap->vetor[indice1 - 1] = heap->vetor[indice2 - 1];
    heap->vetor[indice2 - 1] = aux;
}
```

Heapsort

```
int main()
{
    Heap* heap = cria(10);

    insere(heap, 16);
    insere(heap, 4); insere(heap, 10);
    insere(heap, 14); insere(heap, 7); insere(heap, 9); insere(heap, 3);
    insere(heap, 2); insere(heap, 8); insere(heap, 1);

    heap->tamanho_heap = 10;
    imprime(heap, heap->tamanho_heap); //16 4 10 14 7 9 3 2 8 1
    heapsort(heap);
    imprime(heap, 10); //1 2 3 4 7 8 9 10 14 16
    getchar();
    return 0;
}
```

Heapsort

- Análise de complexidade
 - ▣ Construção da heap
 - $\Theta(n)$
 - ▣ Max_heapify (em $n-1$ nós)
 - $O((n-1)\log(n)) = O(n\log(n))$
 - ▣ Assim, a complexidade de tempo do HeapSort é $O(n\log(n))$

Referências

1. Introduction to Algorithms, Third Edition, Thomas H. Cormen
2. Robert Sedgewick and Kevin Wayne.
2011. *Algorithms* (4th ed.). Addison-Wesley Professional

